

Problem Statement #23: Hyperlocal Courier Dispatch & Tracking Engine

Name : Soumya A

SRN : PES1UG24CS628

Section : K

1. Architecture Selection

We chose Microservices Architecture for the Hyperlocal Courier Dispatch & Tracking Engine. The system is divided into small services (Order, Dispatch, Tracking, OTP and Route Optimizer) that are reached through an API Gateway and communicate only through defined interfaces.

2. Two Scenario-Specific Reasons

1. **Live tracking needs to scale separately.** Every active rider sends GPS updates continuously, but a sender places an order only occasionally. In a microservices design only the Tracking Service is scaled up during peak hours, so the sender gets delivery status in under 2 seconds (NFR-001) without also scaling order handling.
2. **A failure in one part should not stop deliveries.** Multi-stop route optimization is heavy and can be slow or fail. Because it is a separate service, senders can still place orders and the Dispatch Service can still match the nearest rider within 3 km (FR-001). The routing logic can also be updated without touching the OTP or order services.

3. Security Advantage

All requests from the Sender App and Rider App go through the API Gateway, so authentication, HTTPS and rate limiting are enforced in one place and the internal services are not exposed directly. The OTP Service is also kept separate and can be called only by the Order Service. OTPs are stored in hashed form with an expiry time and a limit on attempts, so the delivery verification cannot be bypassed through any other component.

4. Performance Benefit

The Tracking Service keeps the latest rider locations in memory and pushes updates to the sender over WebSocket instead of the sender polling again and again. The Dispatch Service reads these locations through the IRiderLocation interface to find riders inside the 3 km radius quickly. Since this service handles only tracking, its response time is not slowed down by order or routing work.